

Florish Adekogbe

DIFFERENTIAL EVOLUTION OPTIMIZED DEEP NEURAL NETWORK FOR INTRUSION DETECTION SYSTEM

 Quick Submit Quick Submit University of Lagos

Document Details

Submission ID

trn:oid::1:3519483203

Submission Date

Mar 27, 2026, 9:49 PM GMT+1

Download Date

Mar 27, 2026, 10:30 PM GMT+1

File Name

DE_DNN_IDS_Final_Revised_V5.docx

File Size

1.3 MB

37 Pages

7,419 Words

42,779 Characters

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

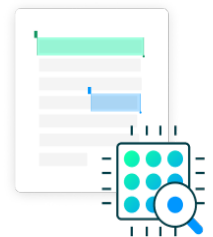
AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.



DIFFERENTIAL EVOLUTION OPTIMIZED DEEP NEURAL NETWORK FOR INTRUSION DETECTION SYSTEM

BY

FLORISH ADEKOGBE

190805025

A PROJECT SUBMITTED TO THE DEPARTMENT OF COMPUTER SCIENCE

FACULTY OF SCIENCE

UNIVERSITY OF LAGOS

SUPERVISOR: DR. B. A. SAWYERR

2026

ABSTRACT

In the last couple of years, the world has seen an exceptional increase in the level of cyber and malware attacks, and this has made it clear that we need more intelligent and adaptive security solutions. Traditional intrusion detection systems frequently find it difficult to keep up with the ever-evolving nature of modern threats. The use of artificial intelligence to create systems that can automatically identify dangers and learn new things is becoming more popular as a result of this reality. The project's objective is to create a Deep Neural Network model that can identify intrusions with high accuracy while minimizing false alarms.

At its core, this research tackles a fundamental challenge in deep intrusion detection to find the right combination of hyperparameters. As researchers have shown in the evolutionary computation literature, metaheuristic algorithms like genetic algorithms and differential evolution offer powerful ways to search through complex spaces where traditional gradient-based methods might get stuck in local optima. The motivation behind this work comes from a simple observation: deep neural networks can be remarkably good at recognizing patterns, but their performance hinges heavily on how we configure their architecture and training parameters.

This approach is based on the Machine Learning Development Lifecycle (MLDLC). It will include gathering and preprocessing data, defining the model architecture, using Differential Evolution to perform hyperparameter optimization, training the model using popular benchmark datasets (NSL-KDD and CICIDS2017), and performance evaluation in detail. Based on the best practices of evolutionary algorithm studies, the DE algorithm will optimize the most significant hyperparameters, including the network depth, the layer sizes, learning rate, dropout rate, and the activation functions, whereas the fitness will be computed based on the model's performance on the validation data.

The desired outcome is a DE-optimized DNN model, which is very efficient in comparison to the standard machine learning classifiers and manually optimized deep learning algorithms on the key metrics related to accuracy, precision, recall, and F1-score.

LIST OF TABLES

Table 2.1: Comparison of Intrusion Detection System Types	10
Table 2.2: Types of Neural Network Architectures	12
Table 2.3: Deep Neural Network Architectures for IDS	14
Table 2.4: Differential Evolution Control Parameters	16
Table 2.5: Comparison of Evolutionary Optimization Algorithms	17
Table 2.6: Summary of Related Works on DE and Neural Networks	20
Table 3.1: Research Environment Specifications	23
Table 3.2: NSL-KDD Dataset Characteristics	24
Table 3.3: CICIDS2017 Dataset Characteristics	24
Table 3.4: NSL-KDD Attack Distribution	25
Table 3.5: Hyperparameter Search Space for DE Optimization	27
Table 3.6: Performance Evaluation Metrics	31

LIST OF FIGURES

Figure 2.1: Taxonomy of Intrusion Detection Systems	10
Figure 2.2: Structure of an Artificial Neuron	13
Figure 2.3: Deep Neural Network Architecture	14
Figure 2.4: Backpropagation Process Flow	15
Figure 2.5: Differential Evolution Algorithm Flowchart	16
Figure 2.6: DE Mutation and Crossover Operations	17
Figure 3.1: Machine Learning Development Lifecycle	21
Figure 3.2: Data Preprocessing Pipeline	25
Figure 3.3: Proposed DE-DNN System Architecture	26
Figure 3.4: DE-Optimized DNN Framework	27
Figure 3.5: Model Training Process Flow	28

TABLE OF CONTENTS

CHAPTER ONE: INTRODUCTION	1
1.1 Background of the Study	1
1.2 Statement of Problem	3
1.3 Aim and Objectives	4
1.4 Scope of Study	4
1.5 Limitation of Study	5
1.6 Significance of Study	5
1.7 Research Methodology	6
1.8 Outline of Project	7
1.9 Definition of Terms	8
 CHAPTER TWO: LITERATURE REVIEW	 9
2.1 Introduction	9
2.2 Conceptual Review	9
2.2.1 Intrusion Detection Systems	10
2.2.2 Machine Learning	10
2.2.3 Types of Neural Networks	11
2.2.4 Deep Neural Networks	12
2.2.5 Evolutionary Algorithms and Differential Evolution	14
2.3 Theoretical Framework	17
2.4 Review of Related Works	18
2.5 Summary of Literature Review	20
 CHAPTER THREE: METHODOLOGY	 21
3.1 Introduction	21
3.2 Research Methodology	21
3.3 Research Environment	22
3.4 Dataset Description	23
3.5 Data Preprocessing	25
3.6 Model Design	26
3.7 Model Training	27
3.8 Performance Evaluation Metrics	29
 REFERENCES	 30-32

CHAPTER ONE

INTRODUCTION

1.1 Background of the Study

The world has been rapidly affected by the spread of digital technologies and explosive increase in the number of internet-connected devices. The digital transformation has presented unbelievable frontiers in the way we communicate, do business and innovations. However, there is another side to the development: it has, too, provided a huge playground for the vicious players that tend to exploit the loopholes of our systems and networks. It is now possible to mention cybersecurity as one of the most burning issues that are faced by organizations, governments, and ordinary citizens in our highly interconnected world.

There are increased cyber-attacks, more advanced malware as well as smarter intrusion attempts than ever before. This rush of threats has resulted in a dire necessity for security solutions not only to be smart, but also able to adapt and learn over time. According to industry reports, the global cost of cybercrime might exceed trillions of dollars every year, and attackers constantly devise innovative and more difficult-to-detect tools. With this scenery, the necessity of superior detection tools capable of identifying not only the established attacks but also new ones using continuous learning and automation with AI becomes apparent.

Intrusion Detection Systems or IDS are now a part of the cybersecurity toolkit. Imagine them as security guards who are continuously monitoring network traffic and system activity and are on the lookout for any suspicious activity that would indicate the presence of a security threat. Historically, these systems have been classified into two broad camps namely signature-based detection and anomaly-based detection. Signature-based IDS operate by matching what it observes against a database that is populated with predefined threat signatures, similar to the operations of antivirus software. Although this method is effective against familiar threats, it just cannot effectively detect a novel or zero-day threat that has never been observed. The anomaly-based method of detection does it differently and learns the look of normal objects and flags anything that does not resemble that normal look. This has the advantage of having

the capacity to detect unknown attacks and it is usually accompanied by the disadvantage of producing excessive false alarms (Garcia-Teodoro et al., 2009).

DNN has demonstrated great success in numerous areas where it is applied in the identification of patterns. Their capability to automatically acquire layered representations over raw data makes them especially suitable for intrusion detection, in which the patterns concealed in the network traffic may be enormously complicated yet incredibly non-linear. Nonetheless, achieving DNNs performance is not an easy task as their usability is highly contingent on the selection of the hyperparameters, such as the network architecture, learning rate, and activation functions.

This is where the evolutionary algorithms come in the picture. It has been found that these algorithms are potent in navigating through complicated optimization landscapes. The studies on the real coded genetic algorithms have revealed that the synergistic optimizations through the global search mechanism and local search methods can significantly enhance optimization outcomes (Sawyerr, Ali, and Adewumi, 2011). Storn and Price introduced Differential Evolution and it received accolades for being easy but efficient in solving non-convex, multimodal optimization problems, without the need to know anything about the gradient.

The proposed research will combine Differential Evolution optimization with Deep Neural Networks to design a more efficient intrusion detection system. Finding the best configurations worldwide, using the capabilities of DE to explore and exploit amidst variations, and relying on the research conducted in the field of evolutionary computation regarding the need to balance the exploration and exploitation phases (Sawyerr, Adewumi, and Ali, 2014), the proposed research will aim to create an IDS that will be capable of the higher detection accuracy and at the same time maintain the number of false positives to the minimum.

1.2 Statement of the Problem

The increasing rate and complexity of cyber-attacks pose immense vulnerabilities to organizations across the globe, endangering the integrity of data and digital infrastructure. This has been due to critical failures in the intrusion detection systems that we currently have which prevent them from being effective in contemporary network environments.

The main issues that this study will be trying to solve are:

1. The signature-based IDS are deeply incapable of detecting new threats, advanced evasion methods and new zero-day exploits with no signature available.
2. The detection systems based on anomaly are more likely to produce a high false positive, thus saturating security teams with false alarms and letting real threats go by.

Although deep learning methods have the potential to be used in intrusion detection, the hyperparameter configuration is significant in evaluating its performance. Manual tuning is time consuming, involves a lot of expertise, and it does not always find those really optimal settings.

The metaheuristic algorithms are effective global optimizers as it has been established in research, however, when not hybridized appropriately, they can be unable to cope with local searches (Sawyer, Adewumi, and Ali, 2014). The classic gradient-based techniques can become trapped at local optima due to the complicated loss landscapes that characterize intrusion detection problems.

The literature on systematic methods to optimize deep neural network architectures and hyperparameters to optimize intrusion detection, in particular, has a very significant gap. Even though it is true that evolutionary algorithms were implemented successfully in a large number of optimization problems, this method is not too extensively investigated in the sphere of optimization of deep learning-based IDS, with a comprehensive optimization of hyperparameters.

This research will solve these issues by defining a framework of Differential Evolution-optimized Deep Neural Network that searches optimal configurations methodically.

1.3 Aim and Objectives

The major goal of the study is to come up with a Deep Neural Network architecture that uses Differential Evolution as its optimizer, with a higher detection rate, and lower false positives in comparison with the conventional algorithms.

To reach this goal, the following specific objectives are established:

1. Carry out an elaborate survey of the available intrusion detection techniques, deep learning models and evolutionary optimization algorithms.
2. Build and implement a Deep Neural Network framework that would best suit intrusion detection issues.
3. Create a DN optimization system based on Differential Evolution to optimize DNN parameters, such as network structure, learning rate, dropout rate, and network activation functions.
4. Combine the DE optimization algorithm with DNN model to allow auto optimization of hyperparameters in the training process.
5. Evaluate the performance of the proposed DE-optimized DNN model using common intrusion detection benchmark data sets, i.e. NSL-KDD and CICIDS2017.
6. Compare the performance of the suggested model to the current machine-based intrusion detection models and deep learning-based intrusion detection models.
7. Evaluate the effectiveness of the DE optimization in improving the detection accuracy as well as false positive rate, using the correct statistical indicators.

1.4 Scope of the Study

This study intends to research on network-based intrusion detection with the use of anomaly-based detection schemes. The system would be capable of binary classification (differentiating between normal traffic and attacks) and multi-class classification (differentiating between particular types of attacks).

To conduct an analysis, the NSL-KDD and CICIDS2017 benchmark datasets are used. It is only applicable to supervised forms of learning with offline batch processing.

1.5 Limitation of the Study

Although this study is relevant to the discipline, it is necessary to note that it has several limitations:

1. The DE optimization is computationally intensive. Since every fitness assessment involves training a full DNN, the nested optimization framework is very resource-intensive in both computing resources and time.

2. The assessment is based on benchmark data (NSL-KDD, dating back to 1999, and CICIDS2017) which might not fully reflect the level of diversity and complexity of network environments in reality.
3. This paper is dedicated to the offline batch processing and mentions nothing about the challenges of the real-time streaming detection in the scenario where latency and the current adaptation are of paramount importance.
4. The proposed system assumes that the training data is labeled, which limits its use in the scenario of the absence of this data or in certain cases.
5. The deployment problems such as integration into the security architecture that is already available and the computing requirements of the system in the production servers fall outside the scope of the research.

1.6 Significance of the Study

The study has academic value as it continues the existing work in the interface of evolutionary computation and deep learning, however, with a specific focus on the topic of hybrid optimization algorithms (Sawyer et al., 2011; 2014; 2015). The suggested DE-optimized DNN model can both enhance the accuracy of IDS and minimize the number of false positives in organizations in practice. Methodologically, it offers a structure in applying differential evolution to the optimization of deep learning in security applications, with stringent standards of experimental evaluation set in the context of evolutionary computation studies. Financially, enhanced intrusion detection is able to limit the cost of cyber-attacks to an acceptable extent.

1.7 Research Methodology

This study is based on the Machine Learning Development Lifecycle (MLDLC) that offers a systematic method of creating machine learning systems. There are seven phases involved in the methodology:

1. Problem Definition: The intrusion detection classification problem was clearly defined, and the performance objectives were established as well as defining performance metrics that would be used in security applications.

2. **Data Collection:** Obtaining the NSL-KDD data set supplied by the Canadian Institute of Cybersecurity and CICIDS2017 data set to facilitate the assessment of a variety of attack instances.
3. **Data Preprocessing:** Data cleaning procedures, one-hot encoding of categorical features, min-max scaling of numerical features, and class balancing with SMOTE (optional).
4. **Model Design:** Construction of the DNN architecture with tunable layers, a searching space of the hyperparameter, and the implementation of the DE optimization system with suitable control parameters.
5. **Model Training:** This model teaches DNNs with Adam optimizer, early stopping, and fitness based on validation set performance, and does the DE optimization with generational running.
6. **Model Evaluation:** The overall evaluation of the performance based on the metrics of accuracy, precision, recall, F1-score, and AUC-ROC on the held-out test data and the statistical analysis of the results.
7. **Results Analysis:** The investigation of the experimental results, comparison with the baselines, conclusions, and recording of results.

1.8 Outline of Project

The project report has five chapters with each having a particular purpose:

Chapter Two (Review): Provides an extensive literature survey, including intrusion detection systems, the basics of machine learning, neural network implementations, evolutionary algorithms, including a special focus on differential evolution, and related literature on the topic.

Chapter Three (Methodology): The description of the ML DLC framework, the description of the research environment, the description of the datasets, the description of the preprocessing methods, and description of model design and training procedures.

Chapter Four (Implementation and Results): Discusses the implementation and presentation of how the proposed system was implemented, and the experimental results gained after the evaluation.

Chapter Five (Summary, Conclusion and Recommendations): Summarizes findings of the research done, concludes the results and gives recommendations on future studies in the field.

1.9 Definition of Terms

Intrusion Detection System (IDS): A security system that monitors system activity and network traffic and looks for indications of program or malicious code abuse, illegal access attempts, or security policy violations.

Deep Neural Network (DNN): This kind of artificial neural network is one where there are several hidden layers between the input and output layers. This depth allows the network to learn hierarchical representations of features from data.

Differential Evolution (DE): A population-based optimization algorithm that uses mutation, crossover, and selection operations to iteratively evolve candidate solutions toward optimal or near-optimal configurations. It was introduced by Storn and Price in 1997.

Hyperparameter: These are the parameters that a practitioner sets prior to commencing the learning process and govern the manner in which the training algorithm operates. These differentiate the model parameters which are learned in training.

Fitness Function: This is the fitness function that is used in the evolutionary algorithms to measure the fitness of the candidate solution. It directs search to optimum settings depending on objectives of the problem.

Machine Learning Development Lifecycle (ML DLC): A structured approach to machine learning application development that encompasses all stages, including the problem definition and data collection, model development, and monitoring.

CHAPTER TWO

LITERATURE REVIEW

2.1 Introduction

This chapter takes a close look at the available literature on the subject of intrusion detection systems, machine learning, neural networks, and evolutionary optimization algorithms. The objective is to put a strong theoretical foundation on it, focus on what other researchers have already done in similar fields, and understand the gaps that the research intends to address. A particular focus is placed on evolutionary computation methods and the way they have been used to solve optimization problems, based on the existing work on hybrid evolutionary algorithms.

2.2 Conceptual Review

2.2.1 Intrusion Detection Systems

IDS have become the necessary building blocks of the contemporary cybersecurity infrastructure. They are automated watchdogs and they are always on top of the network traffic and activities of the system to find out anything that is suspicious. The concept of intrusion detection was initially developed in the early 1980s when Anderson (1980) presented a set of principles that should be used to detect unauthorized computer access.

There are two primary dimensions through which IDS can be classified: the source of their data and the process of identifying threats. Considering the sources of data, we can discuss Network-based IDS (NIDS) and these are placed in strategic locations on the network to analyze traffic as it passes along and Host-based IDS (HIDS) that is placed on specific machines and analyzes events such as system calls and file changes. In terms of methodology of detection, signature-based systems compare observed patterns with known attack signatures, which is effective against known attacks but unaware of novel attacks. Instead, anomaly-based systems are trained on what normal behavior appears to be and alert on abnormal behavior as possible intrusion. This method has a potential of detecting unidentified attacks, although it is prone to produce false alarms (Garcia-Teodoro et al., 2009; Liao et al., 2013).

Figure 2.1: Taxonomy of Intrusion Detection Systems

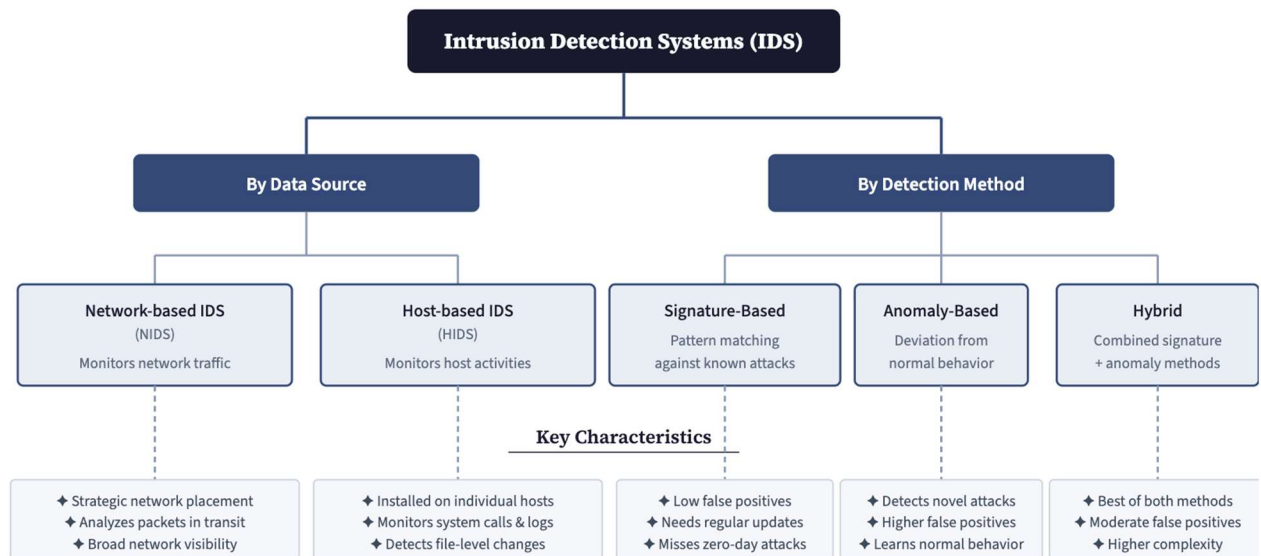


Table 2.1: Comparison of Intrusion Detection System Types

Aspect	Signature-Based	Anomaly-Based	Hybrid
Detection Approach	Pattern matching	Deviation detection	Combined
Known Attack Detection	Excellent	Good	Excellent
Unknown Attack Detection	Poor	Good	Good
False Positive Rate	Low	High	Moderate
Maintenance Effort	High (constant updates)	Moderate	High

2.2.2 Machine Learning

Machine learning is an academic discipline that aims at allowing systems to enhance their functionality through the learning of data instead of using an explicit set of instructions. It is usually classified into three key categories depending on the way the learning process is conducted.

In supervised learning, we use labeled data to train models, i.e., the expected output is known. It is in the aim of learning a mapping that will be able to predict the results accurately with new inputs. Unsupervised learning, conversely, works with unlabeled data and seeks to identify

elusive structures or patterns in the data. Reinforcement learning is a form of learning that involves the interaction of an agent with an environment in which the agent will learn to make a decision based on the feedback which is in the form of rewards or penalties.

In intrusion detection work, supervised learning is usually the method of choice since in this case there is usually access to labeled data of both normal and malicious traffic.

2.2.3 Types of Neural Networks

Artificial neural networks are of numerous varieties, each being tailored to the specific kind of issue it is expected to address. The following are the key intrusion detection architectures:

1. Perceptron: It is a simple type of Neural Network. It is a single-layer device and is only capable of addressing data that are linearly separable
2. Multilayer Perceptron (MLP): It contains multiple layers which can be fully connected thus enabling it to acquire non-linear relationships. We use backpropagation to carry out training (Rumelhart et al., 1986).
3. Convolutional Neural Network (CNN): It involves convolutional and pooling layers in order to extract spatial features of data. It was originally used to work with images but was also applied in other fields (LeCun et al., 2015).
4. Recurrent Neural Network (RNN): This type of connection is looped providing it with some kind of memory. This renders it appropriate with sequential data in which context is significant (Elman, 1990).
5. Long Short-term Memory (LSTM): A type of RNN which assists the network in learning long-term relationships without forgetting.
6. Autoencoder: Trained to replicate its input, learning a concise representation in the process. Useful as an anomaly detector because of the high reconstruction errors in abnormal inputs (Hinton and Salakhutdinov, 2006).

Table 2.2: Types of Neural Network Architectures

Architecture	Structure	Strengths	IDS Application
Perceptron	Single layer	Simple, interpretable	Limited use
MLP	Fully connected layers	Non-linear learning	Classification
CNN	Convolutional layers	Spatial features	Pattern detection
RNN	Recurrent connections	Sequential data	Temporal analysis
LSTM	Gated recurrent units	Long-term memory	Session analysis
Autoencoder	Encoder-decoder	Unsupervised learning	Anomaly detection

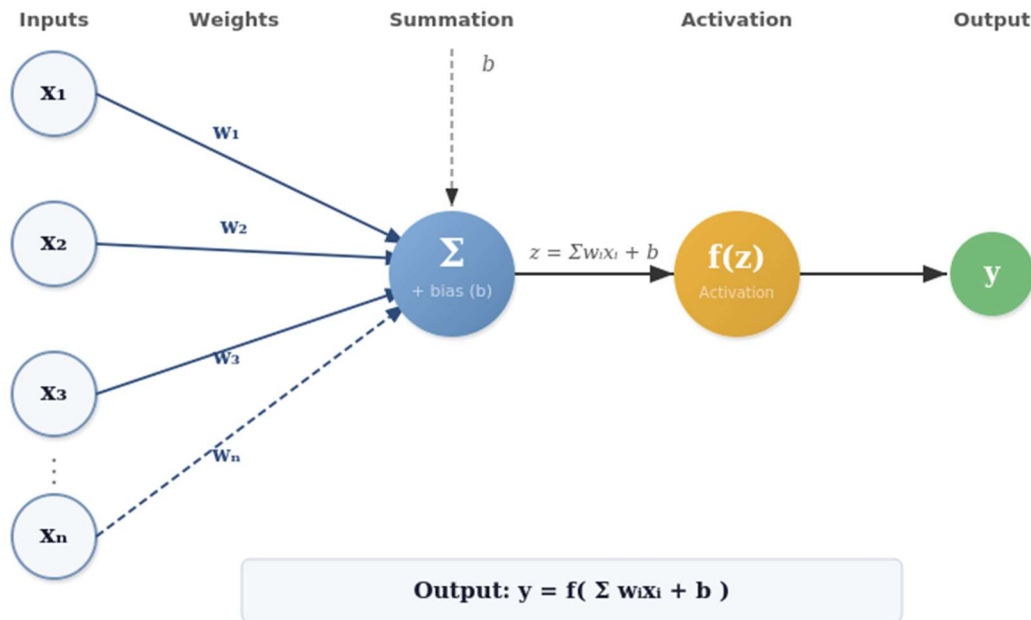
2.2.4 Deep Neural Networks

Deep Neural Networks have multiple hidden layers that make them learn sophisticated features representation of data. These networks are fed with inputs that are transformed in a cascade manner with each layer obtaining progressively more abstract features.

The artificial neuron is at the centre of a neural network and the input values are summed with the weights and bias terms and then an activation function is applied. Activation functions that are often used are ReLU, which adds non-linearity in a simplistic manner, sigmoid, which maps the output to a binary value, and softmax, which gets the output to a multi-class value. Examples of common activation functions are:

1. ReLU (Rectified Linear Unit): $f(z) = \max(0, z)$. It is quite simple and efficient and also prevents the problem of vanishing gradient.
2. Sigmoid: $f(z) = 1/(1 + e^{-z})$. Maps inputs to values between 0 and 1, This works well for binary classification.
3. Tanh: This is similar to sigmoid but it maps to values that are between -1 and 1, which can help with better gradient flow.
4. Softmax: This is used in output layers for multi-class problems, producing a probability distribution over classes.

Figure 2.2: Structure of an Artificial Neuron



Neural network training is the process of maximizing the error reduction through the minimization of the parameters of the neural network. This is done by backpropagation which propagates the error across the network to adjust the weights depending on their contribution to the total loss. The weights are then updated with the following formula: $w_{\text{new}} = w_{\text{old}} - \eta \times \partial L / \partial w$, where η is the learning rate (Rumelhart et al., 1986).

Figure 2.3: Deep Neural Network Architecture

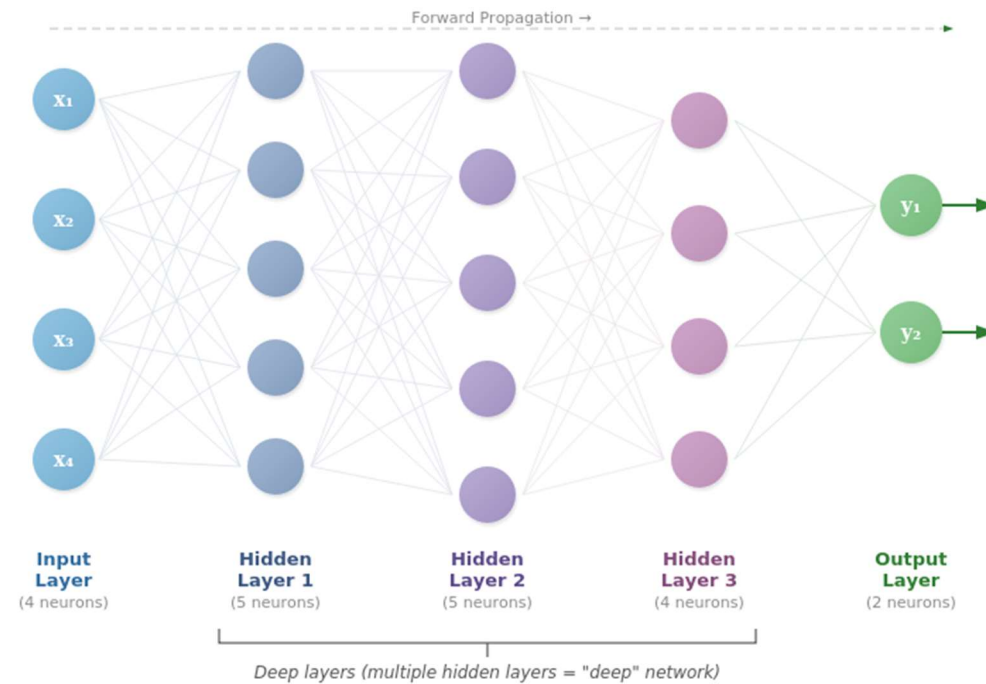


Table 2.3: DNN Architectures Commonly Used in Intrusion Detection

Architecture	Key Characteristics	Typical IDS Application
Multilayer Perceptron	Fully connected feedforward	General attack classification
Deep Belief Network	Stacked restricted Boltzmann machines	Unsupervised feature learning
Convolutional NN	Spatial feature extraction	Pattern recognition in traffic
Recurrent NN / LSTM	Sequential/temporal processing	Session-based detection
Autoencoder	Unsupervised reconstruction	Anomaly detection

2.2.5 Evolutionary Algorithms and Differential Evolution

Evolutionary algorithms (EA) are optimization algorithms that are motivated by biological evolution. They work on the basis of maintaining a pool of candidate solutions that are selected, crossed over and mutated across generations. It has been proved that evolutionary algorithms along with the local search are highly likely to optimize the performance of optimization by establishing a more suitable balance in search and exploitation. Uniform random local search genetic algorithms that have been coded in real code have been particularly useful in continuous optimization problems (Sawyer, Adewumi, and Ali, 2014).

The difference evolution strategy is what is unique with the Differential Evolution (DE) introduced by Storn and Price in 1997. DE, unlike traditional crossover, forms new candidates by initiating scaled variation in the members of the existing population. This algorithm works based on three primary steps:

1. **Mutation:** Creates a donor vector using the formula $v = x_{r1} + F \times (x_{r2} - x_{r3})$, in this case F is a scaling factor that controls how big the mutation steps are.
2. **Crossover:** Hybridizes the donor and the target vectors with a probability of crossover CR , producing a trial vector.
3. **Selection:** It makes use of greedy selection, when the trial vector is better than the target, it substitutes it, otherwise the target is put on to the next generation.

This straightforward mechanism turns out to be remarkably effective at exploring complex search spaces (Das & Suganthan, 2011).

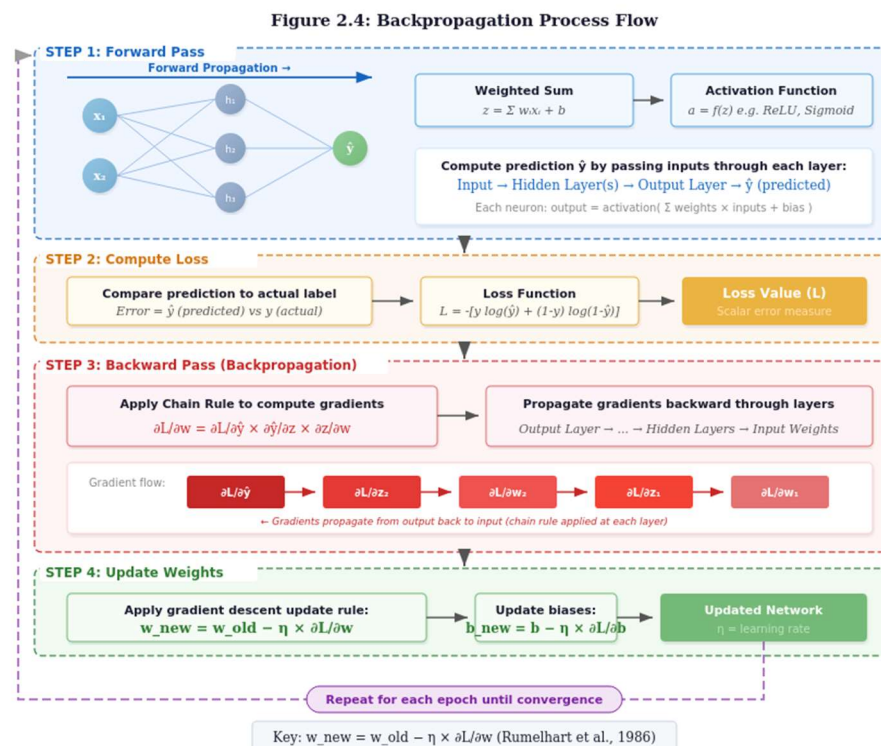


Figure 2.5: Differential Evolution Algorithm Flowchart

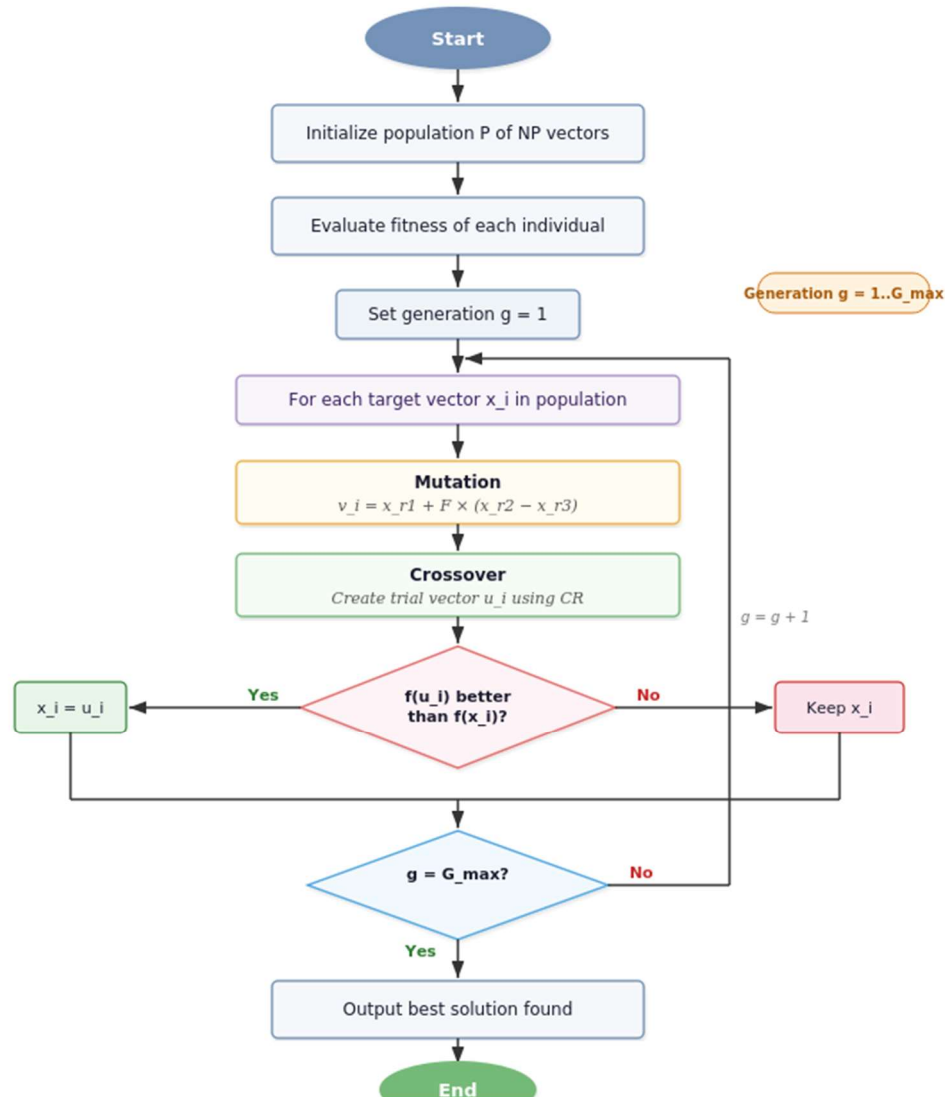


Table 2.4: Differential Evolution Control Parameters

Parameter	Symbol	Typical Range	What It Does
Population Size	NP	5D to 10D	Larger values increase diversity but cost more
Scaling Factor	F	[0.4, 1.0]	Controls how big the mutation steps are
Crossover Rate	CR	[0.1, 0.9]	Higher values create more varied trial vectors
Max Generations	G_max	Problem-specific	When to stop the search

Figure 2.6: DE Mutation and Crossover Operations

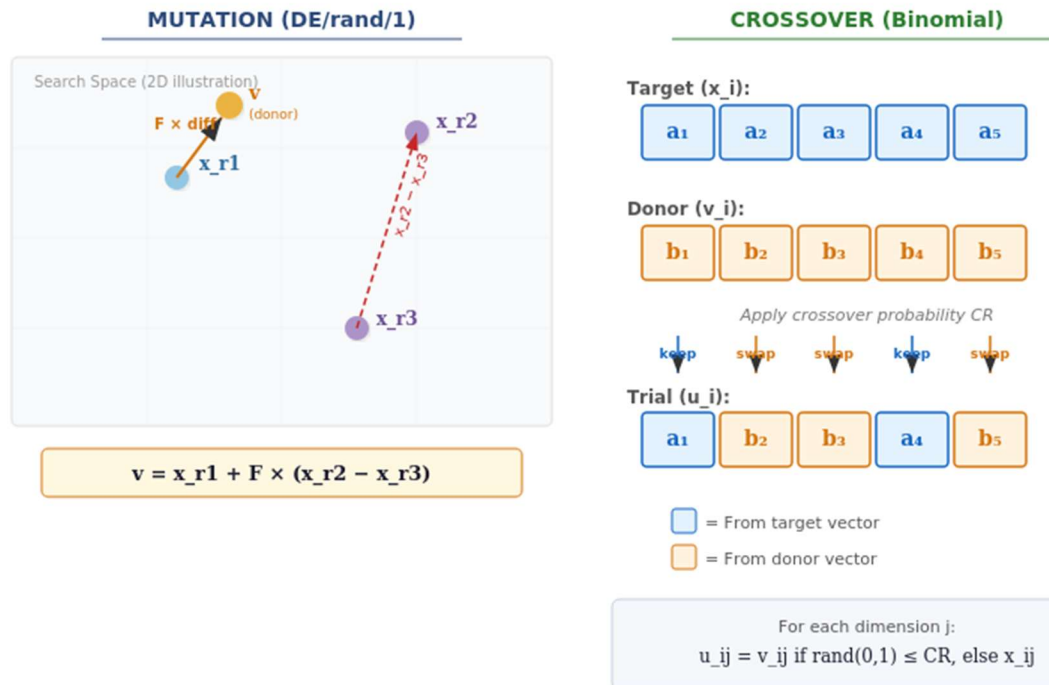


Table 2.5: Comparison of Evolutionary Optimization Algorithms

Algorithm	Mutation Strategy	Main Advantage	Key Reference
Genetic Algorithm	Bit/real crossover	Flexible encoding	Holland, 1975
Real-Coded GA	Arithmetic crossover	Works well in continuous spaces	Herrera et al., 1998
RCGA + Local Search	Hybrid approach	Better exploration-exploitation balance	Sawyer et al., 2014
Differential Evolution	Vector differences	Simple yet powerful	Storn & Price, 1997
Particle Swarm	Velocity updates	Fast convergence	Kennedy & Eberhart, 1995

2.3 Theoretical Framework

The theoretical background of the given research is combined with the concepts of evolutionary computation and deep learning optimization. This confidence can be in the shape of the Universal Approximation Theorem which states that with sufficient capacity, DNNs can be used to approximate any continuous function. Nevertheless, their application training

involves studying non-convex manifolds of minimization as well as a variety of local minima in which gradient-based training algorithms can readily be caught.

One of the indicators that should be kept in mind is the No Free Lunch Theorem which states that there is no optimal algorithm that can be most effectively applied to all the problems. With that said, in certain types of problems, such as the one of neural network hyperparameter optimization, aware algorithmic structure is an order of magnitude more successful than random guessing. Its ability to balance between exploration of unexplored solutions and exploitation of promising ones has been demonstrated to provide experiments with real-coded genetic algorithm models that local search is not only advantageous to the quality of solutions obtained, but also to how fast an algorithm of interest finds the correct solution (Sawyer et al., 2011; 2014).

Since it is a complete parameterization, every member of the population is linked with a specific hyperparameter configuration of the suggested DE-optimized DNN system. The success of the DNN having the structure is evaluated on validation data and the fitness function is used to train the DNN. By the action of the greedy selection, as well as the differential mutation process, the population then changes its structure as a result of the distribution of the difference in mutation and selection to the processes of DE, so as to become structures that would be more efficient in their detection work without falling into the trap of premature convergence.

2.4 Review of Related Works

The section carries out a review of available literature on the topic of deep learning-based intrusion detection and evolutionary optimization of neural networks with a particular emphasis on studies that apply metaheuristic optimization techniques.

Zhang et al. (2019) came up with a deep belief network (GA-DBN) optimized with genetic algorithms to detect intrusion. Their GA-DBN performed better than a standard DBN when experimented on the NSL-KDD dataset. Nevertheless, they did not attempt to search a wider range of hyperparameters and did not consider hybrid methods of combining global and local methods.

Vinayakumar et al. (2019) conducted a wide comparison between DL models used in fIDS, namely conventional DNNs, CNNs, and CNN-LSTM hybrids. They obtained detection rates of more than 98% using NSL-KDD and UNSW-NB15. The weakness of their study was that all hyperparameters were optimized by hand and all possible performance gains due to automated optimization were not put to the test.

Kasongo and Sun (2020) used a feedforward DNN model along with a filter-based feature selection procedure and reached an impressive 99.95% on NSL-KDD. Although they worked, they did not optimize their network architecture but relied on some fixed architecture, which implies that they could have missed even more effective ones.

In the context of the larger evolutionary computation community, studies of hybrid real-coded genetic algorithms have demonstrated that it is much more effective when population-based global search is combined with local search methods. Sawyerr, Ali, and Adewumi (2011) came up with modified crossover procedures building on pattern search methods and performing more successfully on benchmarks. They have since in subsequent work hybridized uniform random local searches (Sawyerr, Adewumi, and Ali, 2014) to further increase the fine-grained search capability. These methods were confirmed by benchmarking studies (Sawyerr, Adewumi, and Ali, 2015), defining stringent assessment practices with regard to the evolutionary algorithms.

The impression that this review leaves is that there is a gap in the problem: no effort has been made to comprehensively optimize hyperparameters of DNNs in intrusion detection. The majority of available research either selects architectures manually or optimizes a small number of hyperparameters. Combining DE optimization with the DNN to IDS and also applying the insights learned in the studies of hybrid evolutionary algorithms is an opportunity and this study provides the answer.

Table 2.6: Summary of Related Works on Deep Learning and Evolutionary Optimization

Authors (Year)	Method	Domain	Key Contribution	Limitation
Zhang et al. (2019)	GA-DBN	IDS	GA optimizes DBN params	Limited hyperparameters

Vinayakumar et al. (2019)	CNN-LSTM	IDS	98%+ detection rate	Manual tuning only
Kasongo & Sun (2020)	DNN + Feature Selection	IDS	99.95% accuracy	Fixed architecture
Baiocchi et al. (2020)	DENN	Classification	DE for NN weights	Not security domain
Sawyer et al. (2011)	Modified RCGA	Optimization	Pattern search crossover	Not applied to DL
Sawyer et al. (2014)	RCGAu	Optimization	Hybrid local search	Not applied to DL

2.5 Summary of Literature Review

This literature review has covered the theoretical backgrounds as well as the empirical research in the field of intrusion detection system, machine learning, neural networks and evolutionary optimization. The main takeaways are:

1. The way intrusion detection systems have evolved is that the system has stopped being operated by basic rule-based systems and instead has undergone advanced deep learning mechanisms that are capable of identifying sophisticated and new threats.
2. Hyperparameters in different architectures of neural networks have been shown to obtain high detection rates (99% and higher) on benchmark datasets, yet the performance of these networks is sensitive to hyperparameter configuration.
3. The evolutionary algorithms and in particular when combined with local search methods are effective at maneuvering through non-convex optimization spaces that are complex.
4. Differential evolution provides a simple and effective population-based optimization model that does not require gradient information.
5. The DE optimization of DNN hyperparameters is a widely-understudied field in particular related to intrusion detection, which this study fills.

CHAPTER THREE

METHODOLOGY

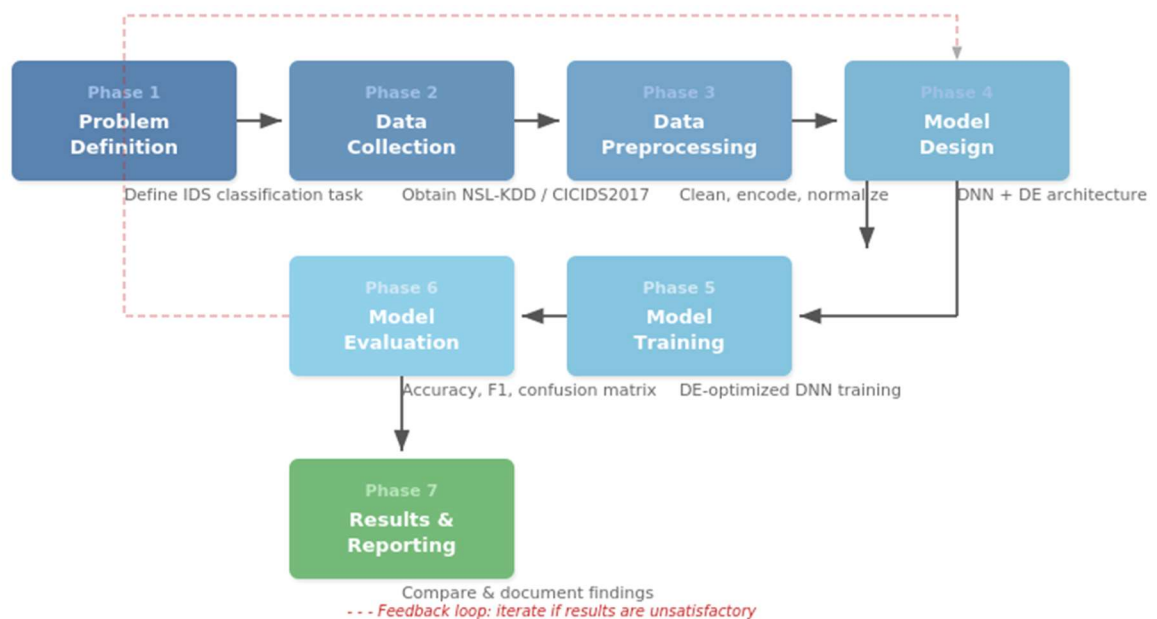
3.1 Introduction

In this chapter, the design and implementation of the Differential Evolution-optimized Deep Neural Network intrusion detection are described extensively. The methodology is based on the machine learning development lifecycle (MLDLC) that guarantees a systematic process of defining the problem up to the evaluation of the final model. Similarly to the standards of the evolutionary computation research field (Sawyer et al., 2015), performance is evaluated with the help of strict experimental methods.

3.2 Research Methodology

In this research, MLDLC framework is employed because it comprises seven stages that can be systematically utilized to build a machine learning system.

Figure 3.1: Machine Learning Development Lifecycle



All of the phases of the MLDLC have their purpose:

1. Phase 1 - Problem Definition: This is the phase where we define the problem of the intrusion detection classification that is what we wish to achieve, what we desire our performance requirements to be (accuracy exceeding 99.0, false positive rate less than 1.0) and what we desire our evaluation criteria imply for security applications.
2. Phase 2 - Data Collection: we gain access to the NSL-KDD dataset, which is offered by the Canadian Institute of Cybersecurity and CICIDS2017 dataset to get an opportunity to take into consideration the different types and events of attacks.
3. Phase 3 - Data Preprocessing: This step includes the cleaning of the data, encoding categorical features with one-hot encoding, normalizing numerical features with min-max scaling and optionally using SMOTE to address the issue of class imbalance.
4. Phase 4 - Model Design: In this step, the data is cleaned, the categorical features are one-hot encoded, the numerical features are min-max scaled, and the classes may be imbalanced, in which case the option of SMOTE is taken into account.
5. Phase 5 - Model Training: DNNs are trained with early stopping with the Adam optimizer. Validation set The performance of validation sets measures fitness and the DE optimization is run over a specific number of generations.
6. Phase 6 - Model Evaluation: we carefully evaluate performance based on accuracy, precision, recall, F1-score, AUC-ROC on held-out test data, and further supported by statistical analysis.
7. Phase 7 - Results Analysis: Results of the experiment will be discussed and compared to the baseline strategies and the conclusion will be drawn and recorded.

3.3 Research Environment

Evolutionary optimization of deep learning models is computationally expensive to run. The research environment is composed of hardware and software that are selected in order to deal with these needs successfully

Table 3.1: Research Environment Specifications

Component	Specification
Operating System	Windows 11 or Ubuntu 22.04 LTS
Processor	Intel Core i7 (10th Gen or newer) or AMD Ryzen 7
RAM	16 GB minimum
GPU	NVIDIA GTX 1080 or RTX 3060 with CUDA support
Storage	256 GB SSD
Programming Language	Python 3.10 or newer
Deep Learning Framework	TensorFlow 2.x with Keras API
Optimization Library	SciPy (scipy.optimize.differential_evolution)
Data Processing	NumPy 1.24+, Pandas 2.0+, Scikit-learn 1.3+
Visualization	Matplotlib 3.7+, Seaborn 0.12+

3.4 Dataset Description

Evaluation is done using two well-known benchmark datasets. They reflect various time intervals and network intrusion data features, and thus, it is possible to evaluate them more extensively.

The NSL-KDD dataset is a better variant of the original KDD Cup 99 dataset that deals with redundancy problems in the former version. All records of the dataset are the network connections and can be categorised into four groups:

1. Basic features: They consist of connection length, type of protocol, service, flag, and number of bytes in each direction.
2. Content features: These include such characteristics as number of failed logins, user being logged in and signs of suspicious activity.
3. Time-based traffic features: Such measurements include a number of connections and error rates for a period of time.
4. Host-based traffic features: These track patterns related to destination hosts, such as connection counts and service usage.

Table 3.2: NSL-KDD Dataset Characteristics

Characteristic	Value
Number of Features	41 (38 numeric, 3 categorical)
Training Samples (KDDTrain+)	125,973
Test Samples (KDDTest+)	22,544
Attack Categories	4 (DoS, Probe, R2L, U2R)
Individual Attack Types	24 in training set, 38 in test set

A more modern point of view is provided by the CICIDS2017 dataset (Sharafaldin et al., 2018). It was developed by the Canadian Institute of Cybersecurity and includes realistic network traffic data that was obtained in five days. The dataset also contains regular background traffic and other recent attacks which include Brute force, DDoS, Heartbleed, SQL Injection, Infiltration, PortScan, and Bot attacks.

Table 3.3: CICIDS2017 Dataset Characteristics

Characteristic	Value
Number of Features	80 extracted flow features
Total Records	Approximately 2,830,743
Attack Types	8 or more categories across 5 days
Capture Duration	Monday through Friday with realistic traffic patterns

Table 3.4: NSL-KDD Attack Category Distribution

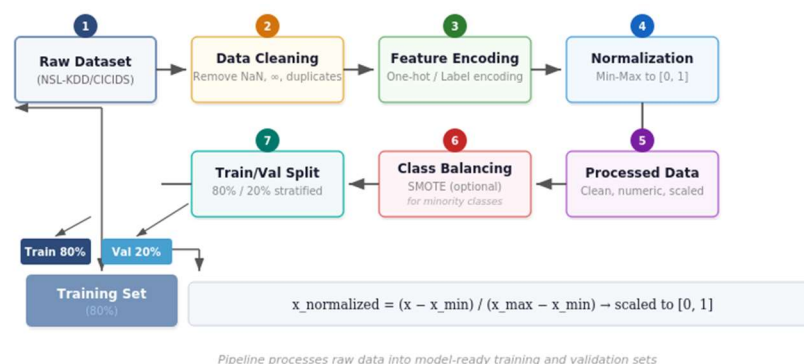
Category	Description	Training Samples	Test Samples
Normal	Legitimate network traffic	67,343	9,711
DoS	Denial of Service attacks	45,927	7,458
Probe	Surveillance and scanning activities	11,656	2,421
R2L	Remote to Local attacks	995	2,754
U2R	User to Root privilege escalation	52	200

3.5 Data Preprocessing

A number of preprocessing tasks will be required before feeding data into the neural network to promote data quality and compatibility. The following tasks are the preprocessing pipeline:

1. **Data Cleaning:** The records containing missing values, infinity values and NaN are eliminated. This is particularly critical to the CICIDS2017 data, which may have values that are problematic.
2. **Feature Encoding:** Categorical features must be changed into numerical ones. In the case of NSL-KDD, the features of protocol type, service and flag are one-hot encoded.
3. **Feature Normalization:** Continuous features are scaled to the [0, 1] range using min-max normalization: $x_{\text{normalized}} = (x - x_{\text{min}}) / (x_{\text{max}} - x_{\text{min}})$. This helps the neural network train more effectively.
4. **Data Splitting:** Stratified data sampling is applied to the data, splitting the data into training (80) and validation (20) sets to ensure that the proportion of classes is not disturbed in the two sets.
5. **Class Balancing:** For severely underrepresented attack categories like R2L and U2R, SMOTE (Synthetic Minority Over-sampling Technique) can optionally be applied to generate synthetic samples.

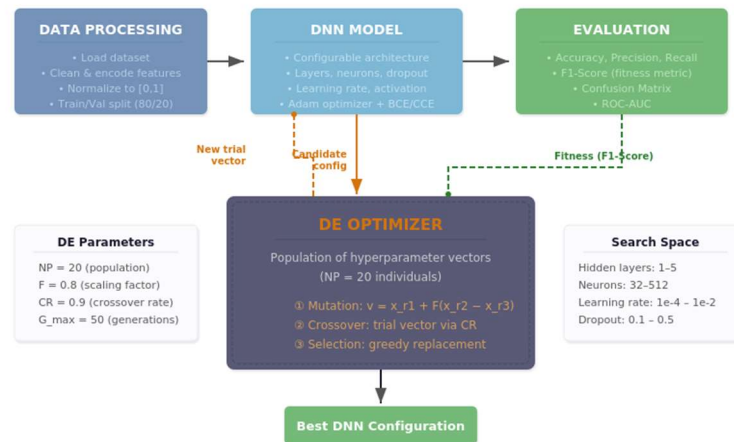
Figure 3.2: Data Preprocessing Pipeline



3.6 Model Design

The proposed system unites the Differential Evolution optimization algorithm and Deep Neural Network training, in a nested system. The outer loop is used to execute the DE optimization over various hypertuning settings and the inner loop to train and evaluate each candidate DNN.

Figure 3.3: Proposed DE-DNN System Architecture



Every member of the DE population is an entire hyperparameter configuration that is represented as a real-valued vector. Continuous values are decoded into integer and categorical parameters when the fitness is calculated.

Table 3.5: Hyperparameter Search Space for DE Optimization

Hyperparameter	Type	Range	How It Is Encoded
Number of Hidden Layers	Integer	1 to 5	Rounded from continuous value
Neurons per Layer	Integer	32 to 512	Rounded from continuous value
Learning Rate	Continuous	0.0001 to 0.1	Log-scale for better coverage
Dropout Rate	Continuous	0.0 to 0.5	Direct continuous value
Batch Size	Integer	16 to 256	Rounded to powers of 2
Activation Function	Categorical	ReLU, tanh, sigmoid	Integer index

Figure 3.4: DE-Optimized DNN Framework

```

Initialize Population: NP = 20 random configurations
↓
For g = 1 to G_max (50 generations):
  For i = 1 to NP:
    1. Pick r1, r2, r3 ≠ i at random
    2. Mutation:  $v = x_{r1} + F \times (x_{r2} - x_{r3})$ , where  $F = 0.8$ 
    3. Crossover: create trial u using CR = 0.9
    4. Decode u → DNN hyperparameters
    5. Build DNN, train for 50 epochs, compute F1-score
    6. Selection: if  $F1(u) \geq F1(x_i)$ :  $x_i = u$ 
  ↓
Return best configuration, retrain for 100 epochs

```

3.7 Model Training

Model training follows a nested approach. A DNN is built and trained to the following specifications, per candidate configuration that DE proposes:

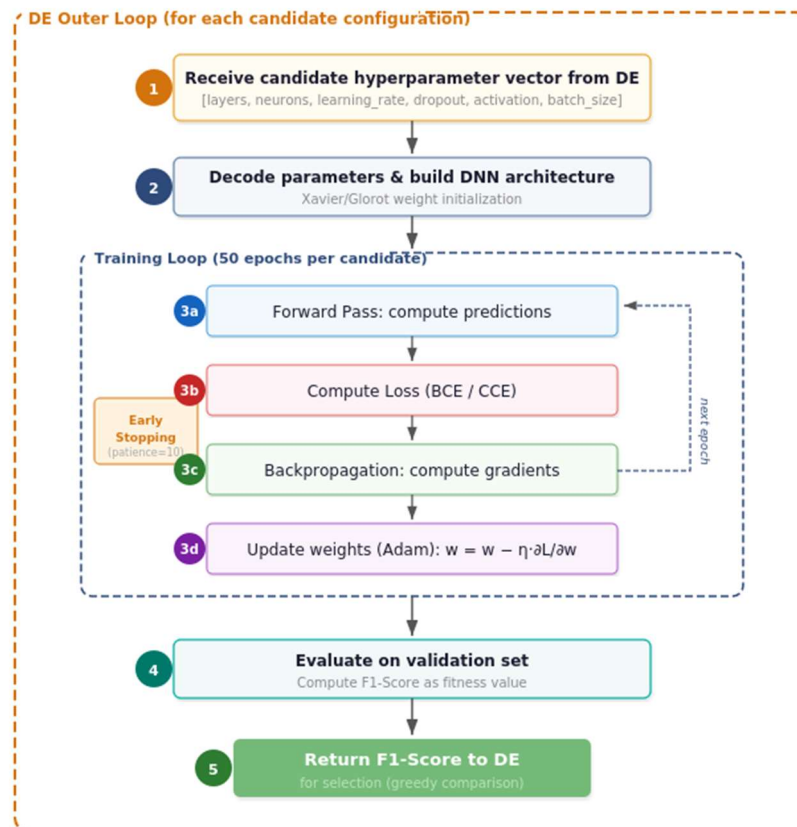
1. Weight Initialization: Xavier/Glorot initialization is used to maintain moderate gradient variance as signals pass through layers.
2. Optimizer: Adam (Adaptive Moment Estimation) will be applied with the learning rate specified by the configuration being evaluated.
3. Training Duration: Each DNN is trained for 50 epochs during fitness examination. The last model, based on the most successful model, is trained for 100 epochs.
4. Early Stopping: If validation loss does not improve for 10 consecutive epochs, training stops early to save time and prevent overfitting.
5. Loss Functions: Binary Cross-Entropy is used for binary classification (normal vs. attack), while Categorical Cross-Entropy is used for multi-class classification.

The DE control parameters are set based on recommendations from the literature:

1. Population size (NP) = 20: This balances having enough diversity with reasonable computational cost.

2. Scaling factor (F) = 0.8: This provides good mutation amplitude for exploring the search space.
3. Crossover rate (CR) = 0.9: A high value encourages variety in trial vectors.
4. Maximum generations (G_max) = 50: This allows enough time for convergence while keeping computation manageable.

Figure 3.5: Model Training Process Flow



3.8 Performance Evaluation Metrics

To thoroughly assess how well the model performs, multiple complementary metrics are used. This follows the rigorous evaluation standards established in machine learning and evolutionary computation research.

Table 3.6: Performance Evaluation Metrics

Metric	Formula	What It Measures
Accuracy	$(TP + TN) / (TP + TN + FP + FN)$	Overall classification correctness
Precision	$TP / (TP + FP)$	How reliable positive predictions are
Recall (Detection Rate)	$TP / (TP + FN)$	What proportion of attacks are caught
F1-Score	$2 \times (P \times R) / (P + R)$	Balanced measure of precision and recall
False Positive Rate	$FP / (FP + TN)$	Rate of false alarms
AUC-ROC	Area under ROC curve	Discrimination ability across thresholds

In these formulas:

1. TP (True Positive): Attacks that the model correctly identified as attacks.
2. TN (True Negative): Normal traffic that the model correctly identified as normal.
3. FP (False Positive): Normal traffic that the model incorrectly flagged as an attack.
4. FN (False Negative): Attacks that the model missed and classified as normal.

For multi-class classification that has many types of attacks, the metrics are calculated separately on a per-class basis, and averaged. With macro-averaging, equal weight is attributed to each class, whereas with weighted-averaging the frequency of classes is taken into consideration.

REFERENCES

- Ali, M. M., Khompatraporn, C., & Zabinsky, Z. B. (2005). A numerical evaluation of several stochastic algorithms on selected continuous global optimization test problems. *Journal of Global Optimization*, 31, 635-672.
- Anderson, J. P. (1980). *Computer security threat monitoring and surveillance*. James P. Anderson Company.
- Baiocchi, M., Di Bari, G., Milani, A., & Poggioni, V. (2020). Differential evolution for neural networks optimization. *Mathematics*, 8(1), 69. <https://doi.org/10.3390/math8010069>
- Das, S., & Suganthan, P. N. (2011). Differential evolution: A survey of the state-of-the-art. *IEEE Transactions on Evolutionary Computation*, 15(1), 4-31.
- Elman, J. L. (1990). Finding structure in time. *Cognitive Science*, 14(2), 179-211.
- Garcia-Teodoro, P., Diaz-Verdejo, J., Maciá-Fernández, G., & Vázquez, E. (2009). Anomaly-based network intrusion detection: Techniques, systems and challenges. *Computers & Security*, 28(1-2), 18-28.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.
- Hinton, G. E., & Salakhutdinov, R. R. (2006). Reducing the dimensionality of data with neural networks. *Science*, 313(5786), 504-507.
- Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735-1780.
- Hornik, K., Stinchcombe, M., & White, H. (1989). Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5), 359-366.
- Kasongo, S. M., & Sun, Y. (2020). A deep learning method with filter based feature engineering for wireless intrusion detection system. *IEEE Access*, 8, 33-44.
- LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436-444.

- Liao, H. J., Lin, C. H. R., Lin, Y. C., & Tung, K. Y. (2013). Intrusion detection system: A comprehensive review. *Journal of Network and Computer Applications*, 36(1), 16-24.
- Mitchell, T. M. (1997). *Machine learning*. McGraw-Hill.
- Rosenblatt, F. (1958). The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6), 386-408.
- Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323(6088), 533-536.
- Sawyerr, B. A., Adewumi, A. O., & Ali, M. M. (2014). Real-coded genetic algorithm with uniform random local search. *Applied Mathematics and Computation*, 228, 589-597. <https://doi.org/10.1016/j.amc.2013.11.097>
- Sawyerr, B. A., Adewumi, A. O., & Ali, M. M. (2015). Benchmarking RCGAu on the noiseless BBOB testbed. *The Scientific World Journal*, 2015, Article 734957. <https://doi.org/10.1155/2015/734957>
- Sawyerr, B. A., Ali, M. M., & Adewumi, A. O. (2011). A comparative study of some real-coded genetic algorithms for unconstrained global optimization. *Optimization Methods & Software*, 26(6), 945-970. <https://doi.org/10.1080/10556788.2010.491865>
- Sharafaldin, I., Lashkari, A. H., & Ghorbani, A. A. (2018). Toward generating a new intrusion detection dataset and intrusion traffic characterization. In *ICISSP* (pp. 108-116).
- Storn, R., & Price, K. (1997). Differential evolution—a simple and efficient heuristic for global optimization over continuous spaces. *Journal of Global Optimization*, 11(4), 341-359.
- Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction* (2nd ed.). MIT Press.
- Tavallae, M., Bagheri, E., Lu, W., & Ghorbani, A. A. (2009). A detailed analysis of the KDD CUP 99 data set. In *IEEE Symposium on Computational Intelligence for Security and Defense Applications* (pp. 1-6).

- Vinayakumar, R., Alazab, M., Soman, K. P., Poornachandran, P., Al-Nemrat, A., & Venkatraman, S. (2019). Deep learning approach for intelligent intrusion detection system. *IEEE Access*, 7, 41525-41550.
- Wolpert, D. H., & Macready, W. G. (1997). No free lunch theorems for optimization. *IEEE Transactions on Evolutionary Computation*, 1(1), 67-82.
- Zhang, J., Zulkernine, M., & Haque, A. (2019). Random-forests-based network intrusion detection systems. *IEEE Transactions on Systems, Man, and Cybernetics, Part C*, 38(5), 649-659.